
The Case For WAL-Logging **pg_upgrade**

Version 2 | Author: Bohyun Lee | patch [GitHub repo](#)

Problem of stock pg_upgrade. Current **pg_upgrade** writes the files of the new cluster directly, leaving no WAL for the changes it makes. This creates two fundamental limitations.

Replication gap. Since the upgrade never enters the WAL stream, standbys cannot adopt it through replication and must be taken offline and resynchronized from the primary via `rsync` [1]. Additionally, if there are cascaded standbys connected to the primary, then the operator must handle them independently.

Durability gap in the backup chain. The upgraded cluster begins with a new WAL history that is disconnected from the history of the old cluster. Consequently, if the upgraded primary fails before a new base backup is taken, transactions committed after the upgrade cannot be recovered. This window may remain open for a long time, if the database is large and taking a backup requires a long time.

Closing replication and durability gaps. The patch closes both gaps by atomically WAL-logging the after-images generated by **pg_upgrade** upon successful completion. The upgrade becomes part of the WAL stream and can be propagated to standbys through standard streaming replication, eliminating the need for offline `rsync`-based resynchronization. Furthermore, because the upgraded cluster continues the existing WAL history rather than creating a new one, the upgrade and all subsequent modifications remain recoverable through WAL replay, preserving durability even before a new base backup is taken.

Upgrade workflow with primary/standby with the proposed interface. (1) Once the old-cluster primary is shut down, (2) **pg_upgrade --wal-upgrade** performs the upgrade (3) and *captures it as WAL*. (4) The primary goes live on restart with the new version and the new cluster standby is also brought up with its retained old data directory. (5) Then each standby streams and replays the upgrade WALs (6) and becomes a hot standby once the WALs are fully replayed.

How primary upgrades cascaded standbys with the proposed interface. (1) During **pg_upgrade --wal-upgrade**, right before the old-cluster primary shuts down, it streams the HANDBOFF record down the cascaded standbys, with each parent waiting only for its direct child's durable acknowledgment. (2) **pg_upgrade** builds the new cluster (3) and it captures the modification as WAL. (4) The primary goes live on restart with the new version. (5) Each standby comes up on the new version and streams and replays the upgrade WALs from its

parent, each parent brought up before its children. (6) Each becomes a hot standby once the WALs are fully replayed, and the configured slots can be used after that point, leaf-first.

Post-upgrade recovery workflow with *primary/backup* with the proposed interface. (1) Restore the last pre-upgrade base backup. (2) PITR-replay archived WAL up to the upgrade point. (3) Restart on the new version and replay the upgrade WAL. (4) Keep replaying the new-version WAL to recover the post-upgrade transactions. (5) Promote, fully recovered without new base backup.

Expected benefits. Upgrading a replicated cluster reduces to upgrading the primary with *significantly less cluster coordination*. Standbys can perform upgrades by restarting once against the new primary and catching up by replaying the upgrade WAL from its retained old data directory — no per-standby rsync or base-backup rebuild. It also guarantees durability of post-upgrade modifications *even without a new base backup*. Such benefits suggest that perhaps `--wal-upgrade` should even be the *default behavior* for the clusters with strict backup/recovery requirements.

1. Proposal Overview

2. [Proposed Interface and How to Use It](#)
3. [Primary-standby Workflow with the New Interface](#)
4. [Primary-standby-standby Workflow with the New Interface](#)
5. [How `--wal-upgrade` Enables Continuous Archiving](#)
6. [Upgrade Storage Validation](#)
7. [Added TAP Tests](#)

2. Proposed Interface and How To Use It

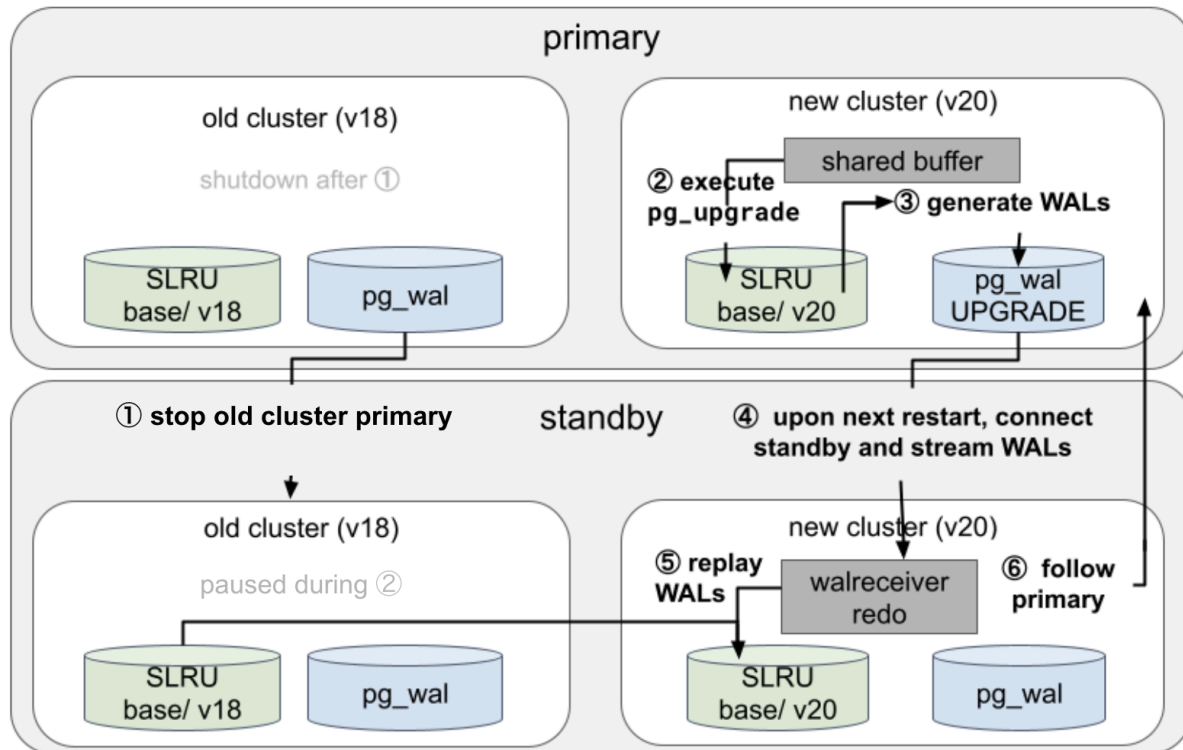
`pg_upgrade --wal-upgrade -d <old> -D <new>` This command is invoked like the vanilla `pg_upgrade`, using the same old and new data directory arguments and supporting the same transfer modes. It produces a fully upgraded cluster on disk and leaves it shut down. Additional step this subcommand adds is that the upgraded cluster backend generates upgrade WALs to capture the upgrade modification. Consequently, on the next restart, the upgraded cluster starts normally in read-write mode, while the upgrade WALs and other subsequent WALs are streamed to standbys through standard replication.

Main advantage of `--wal-upgrade` compared to rsync-based standby upgrade. The conventional way to upgrade standbys is the rsync hard-link procedure: upgrade the primary in link mode, keep the whole cluster down, and rsync each standby from the primary so it rebuilds the new hard-link structure without copying bulk data and to follow the primary without diverging. `--wal-upgrade` replaces that manual file sync with ordinary replication. Both move a similar amount of data when the primary upgrades in a reference transfer mode (link, swap, or clone). The user relations are referenced from each standby's retained old data directory rather than streamed, so the transfer scales with catalog size (i.e., relation count) in both cases. The main advantage of `--wal-upgrade` is that it removes the synchronized downtime, the per-standby file sync, and the mandatory post-upgrade base backup.

Two introduced GUCs. The feature adds two standby-side settings, both required upon a restart and both read only when a standby starts with a minimum skeleton.

- `pg_upgrade_standby_old_datadir` (default empty, superuser-only): the path to this standby's retained pre-upgrade data directory. Recovery reads it to derive the replay-start LSN and, under reference transfer modes, to place (inherit) the user relation files from it.
- `pg_upgrade_standby_transfer_mode` (default mirror): how the standby places user relation files while replaying the upgrade WALs. `mirror` reproduces the transfer mode the primary used, whereas specified `clone`, `copy`, `copy_file_range`, `link`, or `swap` override it locally. For instance, even if the primary performed an upgrade in link transfer mode, the standbys can perform a copy.

3. Primary-standby Workflow with **--wal-upgrade**



① **Stop old-cluster primary.** The operator stops the old-cluster primary. The old cluster standby is still live.

② **Run `pg_upgrade` on primary and pausing standby.** `pg_upgrade --wal-upgrade` runs an ordinary `pg_upgrade`. Right before shutting down the old cluster, the old-cluster primary signals its physical standbys to pause by streaming the `HANDOFF` record to the standby. The paused old cluster standby can still serve read queries.

③ **New-cluster primary emits the upgrade WALs.** To make the upgrade atomic, the new-cluster primary backend runs a single upgrade transaction and emits a set of upgrade WALs (i.e., upgrade window):

- *START* indicates the beginning of upgrade
- *full-page-image* WALs are used for the recreated system catalog relations
- the SLRU and `pg_control`, version files are captured as *RAWFILE* records
- a *RELINK* record maps how each relations/files are operated, whether they are deleted, recreated, inherited from the old cluster, newly created, etc. This varies depending on the transfer mode
- *COMPLETE* finalizes the upgrade after validating the full upgrade correctness. Refer to Section [6. upgrade storage validation](#) for more details how the validation is done.

The upgrade transaction commits, then stops cleanly and writes the completion shutdown checkpoint. The generated upgrade WALs are retained by the inactive migrated slot, pinned at the replay-start LSN from creation until the standby activates it.

④ **New-cluster primary restarts, new-cluster standby ingests the upgrade WALs.** On the operator's next start, the new-cluster primary comes up read-write. The operator stops the paused old standby and starts a fresh new-version standby from a skeleton. The new cluster standby starts with the right configuration, and connects with `primary_conninfo`, derives the replay-start LSN locally from its retained old checkpoint from the old datadir, and streams the upgrade WALs from the primary.

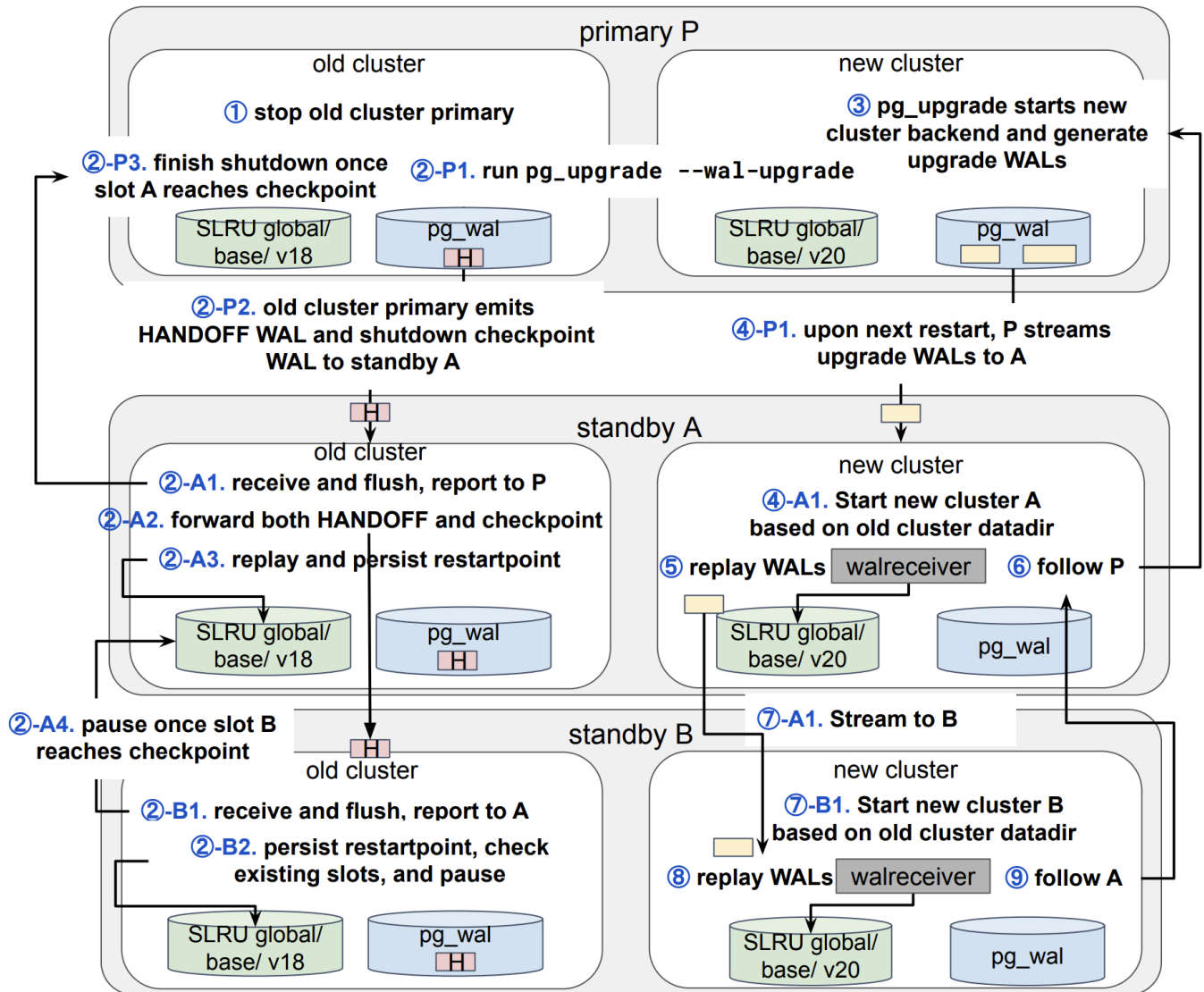
⑤ **New-cluster standby replays the upgrade WALs.** The upgrade WALs are replayed depending on the record type and the configured standby transfer mode:

- *START* checks the new-cluster format and the old source's identity against the replay boundary, binds the retained old datadir, and marks replay started.
- *RELINK* prepares directories and relation forks. In clone, link, and swap modes, inherited user- relation files are respectively cloned, hard- linked, or moved from the retained old standby directory. In copy and copy-file-range modes, relations are marked for recreation instead.
- Standard SMGR and full-page-image WAL records recreate upgraded catalog relations. They also recreate user relations in copy and copy-file- range modes.
- *RAWFILE* records restore `PG_VERSION`, `pg_filenode.map`, `pg_control`, and the SLRU files.
- *COMPLETE* then marks completion pending. The matching *transaction commit* makes the window complete and advances the standby's durable recovery state.

⑥ **New-cluster standby's upgrade finalization.** On replaying the completion shutdown checkpoint, the standby persists at its completion restartpoint, marks its control file upgrade-finalized, and deletes the `pg_upgrade.signal`, and becomes a hot standby. It follows the primary slotlessly, until the operator switches its walreceiver to the named slot.

4. Primary-standby-standby Workflow with the New Interface

Consider a cascade: *primary P* feeds *standby A*, and A feeds its own *standby B*. Each parent feeds its child through a persistent physical slot (slot_a on P, slot_b on A). Here, we focus on how the cascade pauses in lockstep and then adopts the upgrade.



How cascaded standbys are paused

① Stop old primary P, leaving standbys A and B running.

②-P Run `pg_upgrade` and signal handoff to the direct standby. `pg_upgrade --wal-upgrade` arms the handoff before P's final shutdown, writes the HANDOFF record just before the final shutdown checkpoint, and streams both to A. P finishes shutting down once slot_a shows A has durably received the checkpoint. It waits only for A, never for B.

②-**A A acknowledges P, forwards to B, and pauses behind its own child.** A durably flushes both records and reports its position to P, which releases P's shutdown. It replays the checkpoint, persists its restartpoint, and forwards the HANDOFF and checkpoint to B. A then pauses, but only once `slot_b` shows B has durably received the checkpoint, fsyncing that slot first so an A restart cannot forget B's receipt.

②-**B B durably receives and pauses.** B flushes both records and reports to A. Durable receipt needs no replay, so B sits paused below the handoff. As a leaf it has no child to wait for. At this point the whole cascade is paused at the same boundary, and every parent has proof its direct child is there.

How standbys are upgraded and adopted

③ **P generates upgrade WALs.** P starts a new-cluster backend, emits the upgrade WALs (upgrade start LSN..COMPLETE), and shuts P down. Inactive `slot_a` holds those WALs for A at the upgrade start LSN.

④ **Bring up new P, then new A.** P restarts, skips replay, and comes up read-write. The operator starts new-cluster A as a fresh skeleton over A's retained old datadir, pointed at P with `pg_upgrade.signal` and `pg_upgrade_standby_old_datadir`. A recreates its own child slot `slot_b` at the upgrade start LSN and streams the upgrade WALs from P without a slot.

⑤ **A replays and finalizes.** A applies the upgrade WALs -- catalog and system files from the WAL, user relations placed from A's retained old datadir if reference-based transfer mode is used.

⑥ **A follows primary again.** Once upgrade WALs are fully replayed, it automatically becomes a hot standby following P.

⑦-⑨ **Bring up new B the same way.** The operator starts new-cluster B as a fresh skeleton over B's retained old datadir, pointed at A. A serves it the retained upgrade WALs through inactive `slot_b`, B replays them slotless, and becomes a hot standby following A.

⑩ **Return to named slots, leaf-first.** Once the whole tree has finalized, the operator switches the walreceivers back to their named slots bottom-up -- B onto `slot_b`, then A onto `slot_a` -- so retention is released from the bottom up and ordinary cascading replication resumes $P \rightarrow A \rightarrow B$.

5. How `--wal-upgrade` Enables Continuous Archiving

Assume a deployment consisting of a single primary server, daily base backups, and continuous WAL archiving to separate storage. This scenario is used to illustrate how the stock `pg_upgrade` introduces a durability gap and how the proposed patch eliminates it.

The gap caused during the upgrade process. The upgrade initializes the new cluster with a fresh system identifier and a new WAL history. This breaks WAL/LSN continuity at the upgrade boundary, preventing a base backup taken before the upgrade from being replayed into the upgraded cluster; it can restore only up to the old cluster before upgrade. Until a new base backup of the upgraded cluster is created and archived, no valid backup exists for recovery of the committed transactions processed in the new cluster after upgrade.

How `--wal-upgrade` addresses this.

- ① **Capture.** `pg_upgrade --wal-upgrade` runs the normal upgrade, then captures the after-images of every modification it produced as full-page-image WAL records. These are emitted as replayable WALs on the new cluster's log stream.
- ② **Archive.** All the generated upgrade WALs are archived like any other WAL. The archive therefore holds one continuous replay source spanning the upgrade.
- ③ **Recovery phase 1 (old binary).** On loss of the upgraded cluster, the recovery process can restore the last pre-upgrade base backup and, using `restore_command`, replay archived WAL forward to the upgrade boundary, stopping there without promoting. This recovers every pre-upgrade transaction, on the old version, into the restored data directory.
- ④ **Recovery phase 2 (new-binary phase).** The new-version cluster is started on that same data directory. Detecting `recovery.signal` together with the staged `pg_upgrade.signal` sentinel, it clears the version-compatibility gate and arms upgrade replay. Recovery then continues from the base-backup checkpoint and flows organically through CN and the `UPGRADE_*` window, then keeps applying the archived post-upgrade WAL, recovering the transactions committed after go-live, and promotes.

6. Upgrade Storage Validation

Before the new-cluster primary emits the final upgrade COMPLETE WAL record, it validates that the upgraded cluster's storage is consistent with the pre-upgrade state, so a partial upgrade or a missing or wrong file state can never be committed. This is storage-layout and transfer-mechanics validation. It confirms which relations exist, their file mapping, and that the files are present and correctly sized.

How the pre/post-upgrade state is captured. During the upgrade, `pg_upgrade` records two storage inventories into a single relink file.

- **pre-upgrade (OLD):** the relation and directory inventory read from the old cluster while it is still running, just before it is shut down. Each entry is a bare identity (tablespace and relfilenode).
- **post-upgrade (TARGET):** the relation and directory inventory read from the new cluster after the schema is restored and relation files are transferred. Each entry carries full metadata (relkind, persistence, and whether the file was transferred).

How the RELINK WAL operations are built. The new-cluster primary reads the relink file, validates that each inventory captures storage state pre-/post-upgrade, and diffs the old and new inventories to build a per-relation storage operation. There are four storage operations: a relation is deleted, created, recreated, or inherited (a directory is only created or deleted). To figure this out, we diff two inventories via a *single linear merge-join* over the two sorted streams. It keeps the front relation key of each side, compares them, consumes the smaller, and emits the operation from which side the key is on:

- key in the old inventory only → delete (the relation no longer exists),
- key in the new inventory only → create (a new relation),
- key in both → inherit (a transferred file placed by reference) or recreate (rebuilt from WAL), chosen by whether the file was transferred and the transfer mode.

In pseudocode:

```
None
while (have_old || have_target) {
    if (!have_target || (have_old && old_key < target_key)) {
        emit DELETE(old); /* in old only */
        advance_old();
    } else if (!have_old || target_key < old_key) {
        emit CREATE(target); /* in new only */
        advance_target();
    } else { /* keys equal: present in both */
        emit INHERIT_or_RECREATE(old, target);
        advance_old();
        advance_target();
    }
}
```

What is validated: logical level (storage inventory and manifest) Before building the RELINK WAL records, the primary reads the relink file and validates it. The file itself must be intact. Then, for the relations and directories in each inventory, it checks that:

- entries appear in key order, with no duplicates,
- every relation falls under a directory the inventory declares, and
- every declared directory (except the default) is referenced by a relation.

It also confirms each inventory is complete (the entry count matches what was declared) and that template0 and the shared catalog each appear exactly once (extra check). These are the invariants the merge-join relies on, so the diff runs — and COMPLETE is emitted — only if both inventories pass.

What is validated: physical level (on-disk files), on each standby during replay As the standby replays the RELINK operations, it checks the actual files in its retained old data directory. An inherited relation's file must actually be present and the expected size, so a missing or truncated file fails the replay instead of producing a silently wrong cluster (the optional free-space and visibility-map forks may be absent and are rebuilt). Recreated or newly created relations are rebuilt from the WAL instead.

7. Added TAP Tests

To verify these scenarios, the patch adds three TAP tests and shared test modules under `src/bin/pg_upgrade/t/`. **010_wal_upgrade.pl** validates the durability of the PITR path across the upgrade, along with link mode and the rejection of an unfinished upgrade. **011_wal_upgrade_standby.pl** validates standby replication and cascading workflows. **012_wal_upgrade_validation.pl** validates the upgrade's storage operations and failure handling: migrated-slot safety gates, RELINK operations, upgrade WAL replay, and missing-file detection. The shared modules are `WalUpgradeTest.pm` (helpers) and `WalUpgradeCascade.pm` (the two-level cascade run from 011).

Added TAP test 1: 010_wal_upgrade.pl

Test purpose: Validates primary-side `pg_upgrade--wal-upgrade` execution, continuous archiving, point-in-time recovery (PITR) across the upgrade boundary, link mode transfer mechanics, and crash recovery robustness.

What are tested, what are expected results:

- **PITR across upgrade:** Restoring a pre-upgrade base backup and replaying archived WAL through the upgrade boundary recovers both pre-upgrade and post-upgrade transactions into the new binary without needing a new base backup.
- **Link mode & swap transfer:** Verifies hardlinks and file moves behave correctly during execution and replay.
- **Crash recovery & restartpoints:** Simulates immediate shutdown mid-upgrade to verify WAL replay consistency upon restart.

Coverage: Primary upgrade workflow, archive streaming, multi-phase PITR recovery, file transfer modes, and crash safety.

Workflow:

Phase 1: Setup and upgrade.

1. Initialize an old cluster with `archive_mode=on` and an atomic `archive_command`. Load pre-upgrade test data and take a base backup (`pitr_base`).
2. Run `pg_upgrade --wal-upgrade --initdb` to generate the upgrade WALs. The new cluster inherits the archiving configuration and starts auto-serving.
3. Record the old cluster's final LSN (`old_end_lsn` = upgrade boundary) and inspect `pg_waldump` output for archived WAL segments containing upgrade records.

Execute post-upgrade transactions and wait until all generated WAL segments are archived.

Phase 2: Forcing failure.

Issue an immediate shutdown and remove the new cluster's data directory, simulating complete cluster loss where only the pre-upgrade base backup and WAL archive remain.

Phase 3: PITR Phase 1 (old-binary phase).

4. Restore the pre-upgrade base backup into a fresh target node.
5. Configure `restore_command`, set `recovery_target_lsn = old_end_lsn`, `recovery_target_inclusive = on`, `recovery_target_action = shutdown`, and create `recovery.signal`.
6. Start the node using `pg_ctl` with the old PostgreSQL binary. It replays WAL up to the target boundary and shuts down cleanly without promotion. Verify the log output confirms "shutdown at recovery target".

Phase 4: PITR Phase 2 (new-binary phase).

7. Stage the upgrade WAL segments from the archive into `pg_wal/` on the restored node so local recovery re-anchors at the upgrade start point.
8. Update configuration: remove `recovery_target_*` parameters, retain `restore_command`, set `recovery_target_timeline = latest`, and adjust resource parameters to match the upgraded primary.
9. Touch `recovery.signal` and create an empty `pg_upgrade.signal` sentinel, then start the server with the new PostgreSQL binary.
10. Assert that once `pg_is_in_recovery()` becomes false, both pre-upgrade and post-upgrade rows are present and valid, confirming full recovery without a post-upgrade base backup.

Added TAP test 2: 011_wal_upgrade_standby.pl

Test purpose: Validates standby-side streaming replication, handoff signaling, upgrade WAL replay, cascading standbys, transfer mode overrides, and rollback/abort handling.

What are tested, what are expected results:

- **Standby upgrade via WAL:** Standby streams upgrade WALs from primary, applies relation links, and transitions to hot standby on the new version.
- **Cascading standby upgrade:** Multi-tier standbys (P -> A -> B) receive HANDOFF, pause, replay upgrade WALs sequentially, and resume streaming.
- **Aborted handoff rollback:** Restarting the old primary after handoff allows paused standbys to resume without losing original data.

Coverage: Physical replication slots, handoff protocol, standby replay engine, GUC overrides, cascading topology, and negative test cases.

Workflow:

Phase 1: Baseline old primary and standby setup.

1. Initialize an old primary with streaming replication enabled and start a streaming standby. Verify that the standby is active (`pg_is_in_recovery = t`) and holds consistent pre-upgrade data.

Phase 2: Signal handoff (reversible pause).

2. Execute handoff signaling on the live primary via `pg_upgrade`.
3. The primary writes an `XLOG_UPGRADE_HANDOFF` WAL record and shuts down cleanly. The standby replays the handoff record and pauses at the boundary, remaining online in read-only mode to serve queries.

Phase 3: Upgrade primary and emit WALs.

Run `pg_upgrade --wal-upgrade` on the primary. The new primary starts up read-write while physical retention slots pin the upgrade WAL window for standbys.

Phase 4: Provision new-version standby and replay.

4. Provision a new-version standby skeleton configured with `primary_conninfo`, `standby.signal`, an empty `pg_upgrade.signal` sentinel, and `pg_upgrade_standby_old_datadir` pointing to the retained directory.
5. The standby derives its starting LSN locally, connects to the primary, streams the upgrade WALs, and applies `XLOG_UPGRADE_RELINK` records to link user relation files.
6. Validate local transfer mode overrides via `pg_upgrade_standby_transfer_mode` (testing both default mirror and explicit link overrides).
Assert that the standby catches up, becomes a hot standby (`pg_is_in_recovery = t`), sets `upgrade_finalized` in `pg_control`, and survives subsequent restarts normally.

Phase 5: Abort and rollback handling.

7. Verify that restarting the old primary after a handoff allows paused standbys to resume past the handoff without data corruption if the upgrade is aborted.
8. Verify negative test cases: unconfigured GUCs trigger FATAL errors on startup rather than missing user relation files.

Added TAP test 3: `012_wal_upgrade_validation.pl`

Test purpose: Validates storage inventory generation, relink manifest validation, checksum verification, trailer checks, and corruption detection before emitting COMPLETE WAL records.

What are tested, what are expected results:

- **Manifest integrity verification:** Validates magic headers, frame sequences, payload checksums, and trailer checksums on relink files.
- **Corruption rejection:** Corrupted frames, invalid checksums, or missing relation files cause validation failures before emission of COMPLETE records.

Coverage: Relink file validation, manifest checksums, storage inventory sanity checks, and corruption handling.

Workflow:

Phase 1: Valid manifest verification.

9. Generate valid pre-upgrade and post-upgrade storage inventories during a mock upgrade execution.
10. Assert that the primary validates magic headers, frame order, directory coverage, and trailer checksums successfully before emitting COMPLETE.

Phase 2: Negative and corruption test cases.

11. Inject checksum mismatches, truncated frames, out-of-order keys, and missing directories into the relink manifest.
12. Assert that the primary aborts upgrade transaction and logs explicit validation error messages before emitting COMPLETE.